

Explorations in Computer Vision

Dr Simon Lock

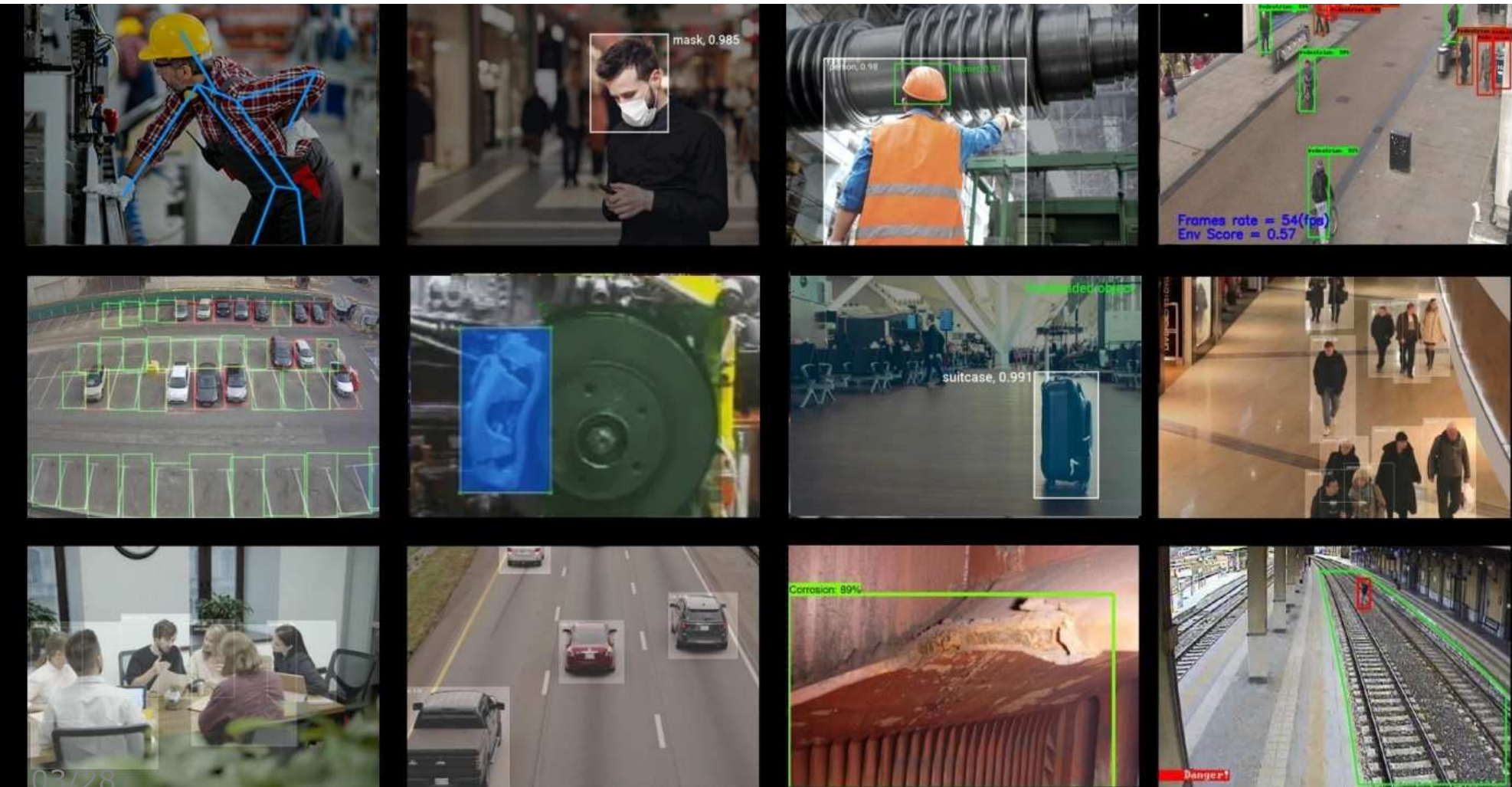
Overview

There are many applications for computer vision
It is powerful to be able to sense our environment
Then make decisions based on live real-time data

Custom sensors can be expensive and inflexible
Cameras and analysis software much more versatile

Let's look at some example applications...

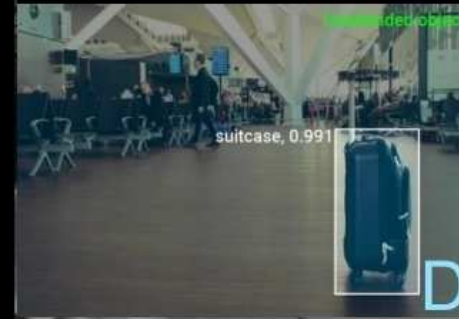
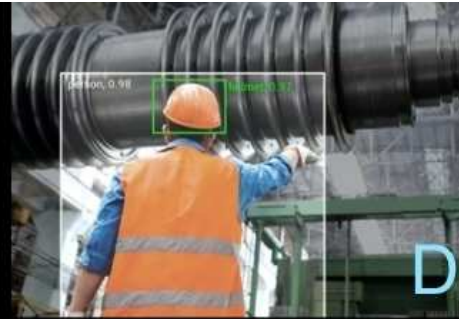
Example Computer Vision Applications



Different Levels of Comprehension

- Attention: is there *something* (anything) present ?
- Detection: specific *type* of thing (car, person etc) ?
- Recognition: which *individual* thing (car, face etc) ?
- Interpretation: what is the thing actually *doing* ?

Classification



Our Approach for this Session

We **could** try to train a machine learning model
In order to identify objects/structures in an image
But this wouldn't give much insight into the process
(We'd just be training, rather than experimenting)

Instead we will take a much lower-level approach
Focusing on writing code to analyse video frames
Using a number of "pixel analysis" techniques...

Pixel Analysis Techniques

- Searching for specific colours (RGB, Hue, Sat. etc.)
 - Frame differencing (change in Brightness, Hue etc.)
 - Location sensing (position of artefacts is important)
 - Movement detection (change in location is useful)
-
- Scanning for specific shapes (people, car, letters etc.)
 - Matching relative structures (e.g. face detection)

The Application

We need some form of visual material to process
For this, we will use a recording of a webcam stream
From the "Marine Biological Association" in Plymouth:

[3m5!1s0x486c935311b11e0b:0xd40b5a4f7f597d10!8m2!3](#)

It's a nice view, with lots of activity going on !

video

Your Task

Shown in the camera stream is a busy road junction

Your task: analyse traffic passing through this junction

Don't panic: step-by-step instructions will be provided !



Processing Template

To get you started, we've provided a template to:

- Load in a video stream
- Extract a single image ("frame") from the stream
- Draw that frame onto the screen
- "Mask off" unwanted areas of the frame
- Define a colour range for analysis
- Analyse "unmasked" pixels within frame

VehicleAnalysisTemplate

Drawing on a Frame

To mask off areas of the frame:

- Set the "fill" colour to black (0,0,0)
- Then draw Rectangles and Triangles

To draw individual pixels on the frame:

- Set "stroke" colour - for example red (255,0,0)
- Then draw individual "points" (pixels) in that colour

Examples of all the above are in the code template

Checking the Colour of Individual Pixels

We can get the colour of a particular pixel using:

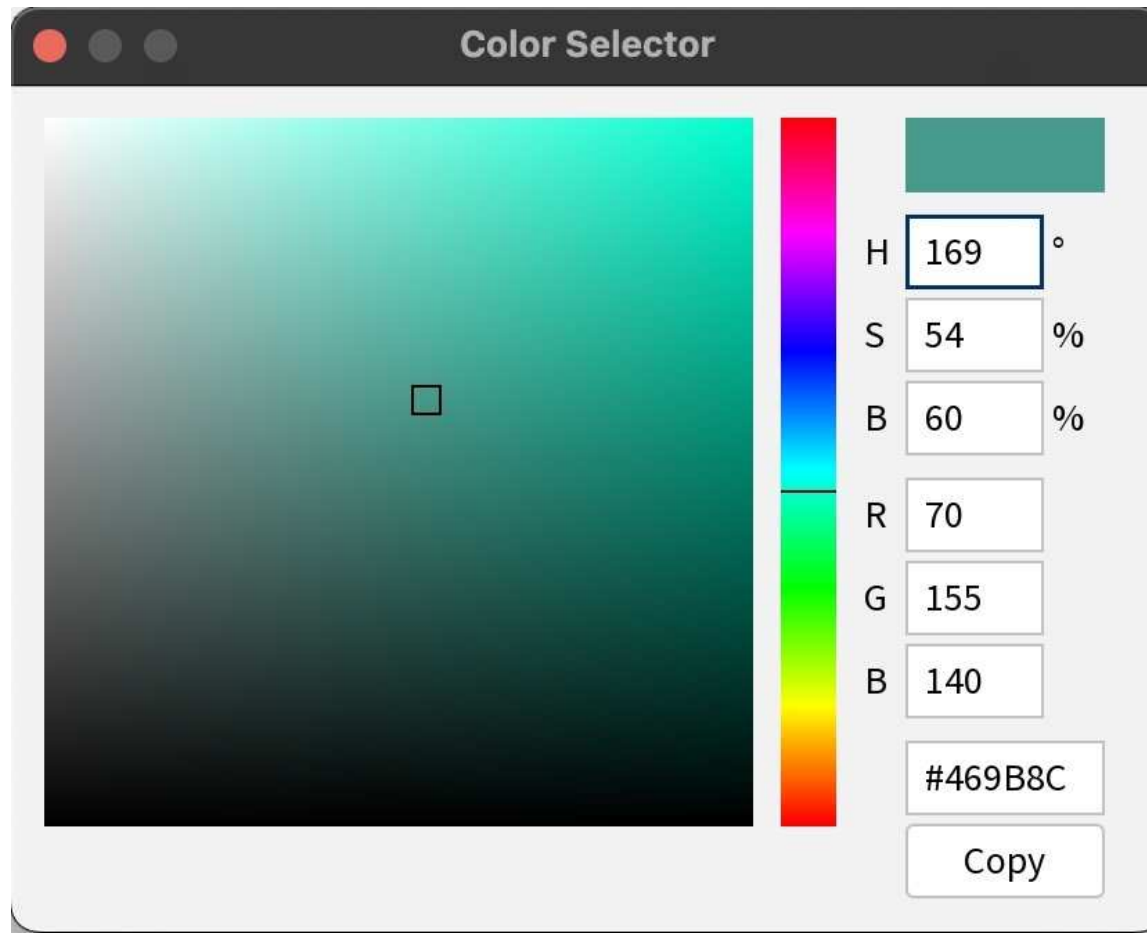
```
int pixelColour = currentFrame.get(x,y);
```

Then extract various properties of the pixel using:

```
int redness = red(pixelColour);  
int greenness = green(pixelColour);  
int blueness = blue(pixelColour);  
int whichColour = hue(pixelColour);  
int howBright = brightness(pixelColour);  
int howSaturated = saturation(pixelColour);
```

Hue / Saturation / Brightness / RGB

Warning: All values in Processing are in range 0-255



IMPORTANT NOTE

This is a very processor-intensive activity !
280k pixels per frame requires a LOT of calculation

If you do not have a fast laptop
We advise you to use a lab machine !!!
(otherwise it will be painfully slow !)

Instructions

Go to the summer school GitHub page:
<https://github.com/UoB-SummerSchool/>

Clone the repository for this exercise:
CameraAnalysisAndComms

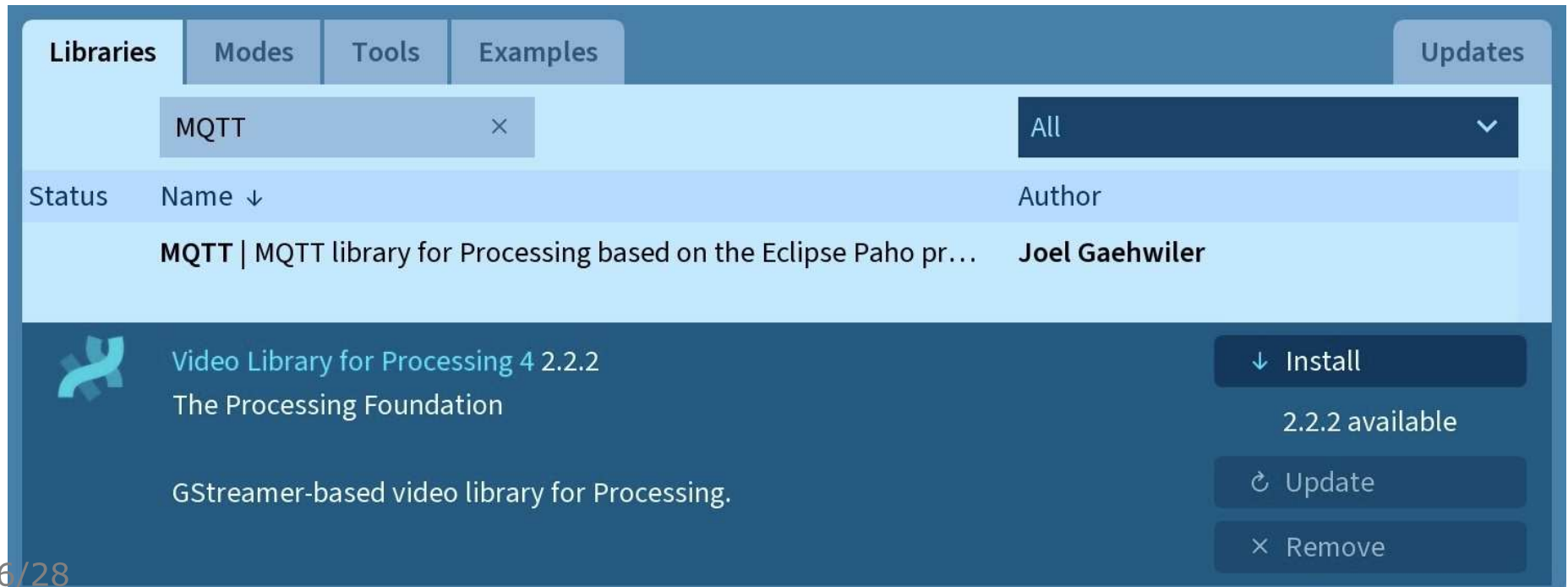
Work through the tasks in the instructions document

Library

To run template project, you'll need to install a library:

Sketch > Import Library > Manage Libraries

Then search for a library called "MQTT" and install



To Work !

Remote Notification

Remote Notifications

It's no good being able to detect something...

If nobody actually realises the event has occurred !

We might even want to notify multiple remote users
(In case many people are interested in the event)

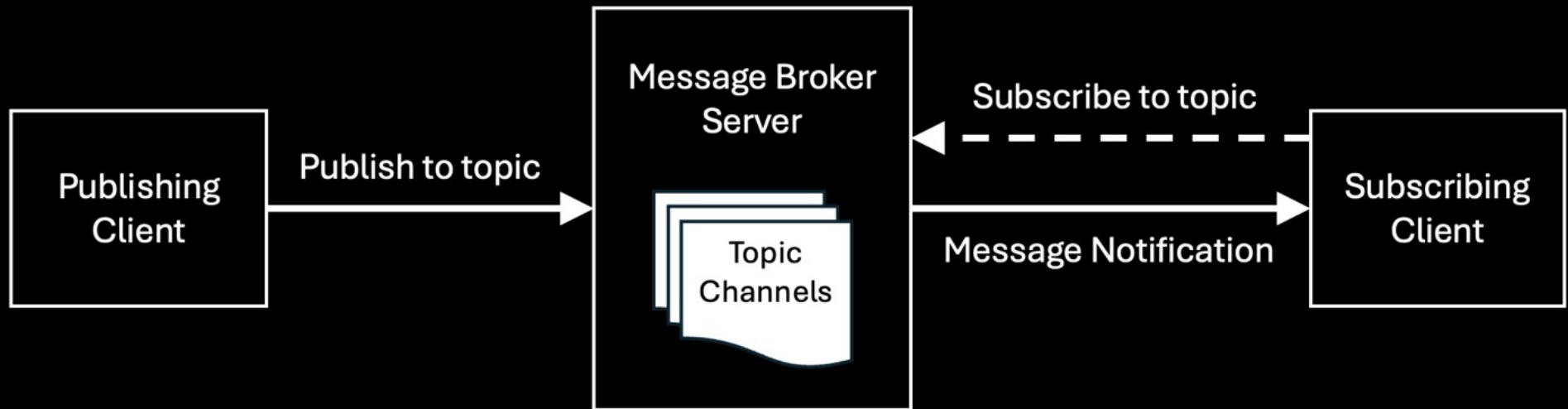
There are various mechanisms available to do this
We use a communication mechanism called MQTT...

Message Queue Telemetry Transport (MQTT)

Message passing protocol (ideal for notifications !)

You can "publish" messages to "topic" (channels)

Other people "subscribe" to receive notifications



Message Queue Telemetry Transport (MQTT)

Connect to broker by adding this to setup method:

```
mqtt = new MQTTClient(this);  
mqtt.connect("mqtt://broker.hivemq.com:1883");
```

To publish a notification message to a topic we use:

```
mqtt.publish(topic, parameter);
```

Topic should start with "junction/" for example:

```
mqtt.publish("junction/vehicles/car/red", "North");
```

Alternative Servers

The server that routes messages is called a "Broker" (Arbitrates between publishers and subscribers)

For this activity we are using a free public broker
Since it is free, there's no guarantee of availability
Sometimes the server is not operational :o(

That's fine - there are other alternatives available:

`mqtt://broker.emqx.io:1883`

`mqtt://broker.hivemq.com:1883`

`mqtt://test.mosquitto.org:1883`

`mqtt://public-mqtt-broker.bevywise.com:1883`

Central Projection Screen

We'll run a message monitor on the projector screen:

`MultiNotificationMonitor`

Any vehicle events published to the brokers will appear

Let's run a test to make sure everything is working:

`NotificationTester`

To Work !